CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE

LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
RELATÓRIO: PRÁTICA 3

Belo Horizonte, 18 de Setembro de 2025

1. Programa ex01.asm

```
    #include <p18f4550.inc>

    org 0x0000
    bra    start

    org 0x0020
start:
    LFSR FSR0, 0x300        ; Carrega FSR0 para banco 3
    LFSR FSR1, 0x400        ; Carrega FSR1 para banco 4
    ; Essa parte define os valores de início do vetor
    CLRF INDF0              ; Limpa 0x300
    MOVFF INDF0, INDF1      ; Cópia 0x300 para 0x400
; loop da primeira parte (preencher banco 3)
loop_1:
    MOVFF FSR0L, PREINC0    ; i + 1 recebe seu próprio indíce
    MOVF FSR0L, W, 0        ; WREG recebe o índice atual (Atualizar Flag)
    BNZ  loop_1             ; Repita até Z = 0
    LFSR FSR0, 0x300        ; Limpa FSR0
; loop da segunda parte (preencher banco 4 com banco 3)
loop_2:
    MOVFF PREINC0, PREINC1  ; Move (i + 1) Banco 3 para (i + 1) Banco 4
    MOVF FSR1L, W, 0        ; WREG recebe o índice atual (Atualizar Flag)
    BNZ  loop_2
out:
    bra out
    end
```

Definição de endereçamento indireto: É um modo de acessar dados na memória onde a instrução fornece um endereço que aponta para uma posição de memória ou registrador, que por sua vez contém o endereço efetivo do operando.
As Vantagens deste tipo de endereçamento e aplicações: Flexibilidade, permite gerenciar e manipular dados de forma mais flexível. Acesso a grandes blocos de dados, sendo útil para acessar matrizes ou estruturas de dados complexas onde o endereço inicial pode mudar dinamicamente.

Ao final do programa, a seguinte tabela de dados é mostrada na Figura 1 e 2:

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
2D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	....
2E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	....
2F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	....
300	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	....
310	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	....
320	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	!"#$%&' ()*+,-./
330	30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F	01234567 89:;<=>?
340	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	@ABCDEFG HIJKLMNO
350	50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	PQRSTUVW XYZ[\]^_
360	60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	`abcdefg hijklmno
370	70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F	pqrstuvw xyz{|}~□
380	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	....
390	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F	....
3A0	A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	....
3B0	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	....
3C0	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	....
3D0	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	....
3E0	E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	....
3F0	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF	....

Figura 1:. Banco 3 totalmente preenchido com valores de 0 a 255 em seus respectivos endereços

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
3F0	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF	....
400	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	....
410	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	....
420	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	!"#$%&' ()*+,-./
430	30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F	01234567 89:;<=>?
440	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	@ABCDEFG HIJKLMNO
450	50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	PQRSTUVW XYZ[\]^_
460	60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	`abcdefg hijklmno
470	70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F	pqrstuvw xyz{|}~□
480	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	....
490	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F	....
4A0	A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	....
4B0	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	....
4C0	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	....
4D0	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	....
4E0	E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	....
4F0	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF	....

Figura 2:. Banco 4 totalmente preenchido com valores copiados do Banco 3

2. Programa ex02.asm

```
    #include <p18f4550.inc>
    ; Valores de cada componente do vetor
    #define V0      0xCD
    #define V1      0x81
    #define V2      0x5E
    #define V3      0x3F
    #define V4      0x01
    ; Endereço de memória de cada componente
    #define F0      0x200
    #define F1      0x201
    #define F2      0x202
    #define F3      0x203
    #define F4      0x204
    ; Endereços de memória de trabalho
    #define ELEITO  0x00

    org 0x0000
    bra    start

    org 0x0020
start:
    CLRF ELEITO         ; Limpar Eleito
    LFSR FSR0, F0       ; FSR0 definido em P0
    MOVLW V0            ; Início: Carregar componentes do vetor
    MOVWF POSTINC0
    MOVLW V1
    MOVWF POSTINC0
    MOVLW V2
    MOVWF POSTINC0
    MOVLW V3
    MOVWF POSTINC0
    MOVLW V4
    MOVWF POSTINC0      ; Fim: Carregar componentes do vetor
    LFSR FSR0, F0       ; FSR0 redefinido em P0
    LFSR FSR1, F0       ; FSR1 definido em P1

init:
    MOVF FSR0L, W       ; W <- FRS0L
    SUBLW 0x04
    BZ  out             ; Condição de parada: FRS0L - 0x04 = 0

execution1:
    INCF FSR0L, F       ; P0 = P0 + 1
    MOVFF INDF0, ELEITO ; Eleito = Num[P0]
    MOVFF FSR0L, FSR1L  ; P1 = P0
```

```
execution2:
    DECF FSR1L, F        ; P1 = P1 - 1

condition1:
    BN init              ; P1 < 0 ?
    MOVF INDF1, W        ; W <- Num[P1]
    CPFSGT ELEITO        ; Skipa se ELEITO > Num[P1]
    bra execution3
    bra execution2

execution3:
    MOVLW 0x01           ; W = 1
    MOVFF INDF1, PLUSW1 ; Num[P1 + 1] = Num[P1]
    MOVFF ELEITO, INDF1 ; Num[P1] = Eleito
    bra execution2
out:
    bra out
    end
```

O seguinte programa ordena um vetor de 5 componentes, cada uma contendo um valor de 8 *bits*, em ordem crescente.

Primeiramente o programa define algumas diretivas importantes para execução: **V0**, **V1**, **V2**, **V3** e **V4** iguais, respectivamente, a *CDh*, *81h*, *5Eh*, *3Fh* e *01h*, serão armazenadas em **F0**, **F1**, **F2** **F3** e **F4**, nessa ordem. Todas essas informações representam, juntas, um vetor de 5 posições, com números desordenados. Além do vetor, utilizaremos o espaço de memória *0x000* para armazenar o valor de uma variável auxiliar denotada por **ELEITO**.

Antes de iniciar o fluxograma de ordenação, limpamos o espaço de memória de **ELEITO** e carregamos o vetor no Banco 2, garantindo que inicializaremos o programa com valores de execução conhecidos. Em seguida carregamos os registradores **FSR0** E **FSR1** para apontarem para o primeiro endereço do vetor (*0x200*).

Para o fluxograma, podemos correlacionar variáveis com alguns registradores:

P0/1 equivale a **FSR0/1**
Números(**P0/1**) equivale **a INDF0/1**

Assim, cada linha do fluxograma pode ser escrito com uma instrução, como exibido nos comentários de cada linha do programa.

Em suma, o programa varre o vetor de trás para frente, comparando o valor observado com um **ELEITO**, quando o valor comparado é menor que o **ELEITO**, aquele se torna o novo **ELEITO** e os valores trocam de endereço de memória (*swap*). A cada iteração o vetor começa a varrer de uma posição mais próxima do final do vetor até uma condição de parada *(i - 4 == 0)*. Com a condição de parada garantimos que a ordenação aconteça entre os endereços *0x00* e *0x04*, caso contrário, o resultado final do programa seria indeterminado.

Ao final do programa, a seguinte tabela de dados ordenados é mostrada na Figura 4:

Address	00	01	02	03	04	05	06
1B0	00	00	00	00	00	00	00
1C0	00	00	00	00	00	00	00
1D0	00	00	00	00	00	00	00
1E0	00	00	00	00	00	00	00
1F0	00	00	00	00	00	00	00
200	CD	81	5E	3F	01	00	00
210	00	00	00	00	00	00	00
220	00	00	00	00	00	00	00

Figura 3:. Banco 2 com vetor desordenado

Address	00	01	02	03	04	05	06
1B0	00	00	00	00	00	00	00
1C0	00	00	00	00	00	00	00
1D0	00	00	00	00	00	00	00
1E0	00	00	00	00	00	00	00
1F0	00	00	00	00	00	00	00
200	01	3F	5E	81	CD	00	00
210	00	00	00	00	00	00	00
220	00	00	00	00	00	00	00

Figura 4:. Banco 2 com vetor ordenado

3.

Para a questão 3 da prática, foi possível criar dois algoritmos, os quais realizam a operação:

$$(Q, R) = \frac{A}{B},\ B \neq 0$$

Onde **Q** e **R** são quociente e resto. A principal diferença entre os programas está no conjunto numérico de A e B. O primeiro programa (*ex03.0.asm*) realiza divisão apenas com números *unsigned*, um caso mais restrito, já o segundo (*ex03.1.asm*) realiza divisão de números *signed* (complemento de 2).

Apresentemos os programas:

Programa ex03.0.asm

```
#include <p18f4550.inc>

#define VA      0x3F    ; Valor de VA
#define VB      0x03    ; Valor de VB
#define FA      0x00    ; Endereço de VA
#define FB      0x01    ; Endereço de VB
#define FQ      0x02    ; Endereço do Resultado (Quociente)
#define FR      0x03    ; Endereço do Resto da divisão
```

```
    org 0x0000
    bra    start
    org 0x0020
start:
    MOVLW VA        ; Carregar VA
    MOVWF FA, 0
    MOVWF FR, 0     ; Carregar FR com Valor de VA
    CLRF  FQ        ; Inicializando - Limpa Quociente
    MOVLW VB        ; Carregar VB
    MOVWF FB, 0

    MOVF  FA, W     ; Verifica se é divisão do tipo: 0/X = 0
    BZ out
    MOVF  FB, W     ; Verifica se é divisão do tipo: X/0 = indf
    BZ  out

    CPFSLT  FR      ; Verifica se FR < VB (fração própria)
    bra sub         ; SIM ? Então começa a operar
    bra out         ; NÃO ? Então vai pra parada

; Loop de subtrações
sub:
    INCF FQ, F      ; Incrementa Quociente para cada VB removida de FR
    SUBWF FR, F     ; Subtrai VB de FR e armazena em FR
    CPFSLT FR      ; Skipa (vai para parada) se FR < VB
    bra sub
out:
    bra out
    end
```

Para esse programa:

$$(Q, R) = \frac{A}{B},\ B \neq 0,\ A \in \mathbb{N},\ B \in \mathbb{N}^*$$

Inicialmente definimos endereços de memória para armazenar o valor de **A**, **B**, **Q** e **R**, respectivamente em **FA**, **FB**, **FQ** e **FR**. Carregamos os valores de entrada **VA** e **VB** na memória e iniciamos as variáveis de saída, limpando **FQ** e carregando **FR** com valor de **FA**.

A ideia de se inicializar programa dessa forma, é incrementar **FQ** a cada vez que for possível remover o valor de **FB** de **FR**, em suma, estamos contando quantas vezes **B** cabe em **A**, o resultado será **Q** e a parte restante será **R.**

Antes de iniciar o *loop* de divisão, foram inseridos saltos condicionais para tratar casos especiais:

- Divisão por zero *(X/0)*: resultado indefinido.
- *VA = 0*: resulta sempre em *(Q, R) = (0, 0)*, portanto o programa é encerrado antecipadamente para evitar processamento desnecessário.

Uma particularidade importante desse programa é que, para valores de **VA** muito maiores que **VB**, o número de iterações pode ser elevado. Por exemplo, com *VA = 0xFF* e *VB = 0x01*, são necessárias 255 iterações para completar a operação. Considerando que as 4 instruções

dentro do *loop* levam aproximadamente 4 pulsos de *clock* cada, cada iteração consome cerca de 16 pulsos, totalizando aproximadamente 4080 pulsos de *clock*. Dessa forma, fica evidente a ineficiência desse método para aplicações mais gerais.

A Figura 5 mostra, ao fim do programa, os endereços de memória com os valores de resultado da divisão de *0x3F* (63) pro *0x03* (3), espera-se que *(FQ, FR) = (0x15, 0) = (21, 0)*:

Address	00	01	02	03	04
000	3F	03	15	00	00
010	00	00	00	00	00
020	00	00	00	00	00
030	00	00	00	00	00
040	00	00	00	00	00
050	00	00	00	00	00

Figura 5:. Em azul, os endereços FQ e FR com os valores esperados

Pensando em contornar a limitação no tempo de execução e com a finalidade de ampliar o conjunto numérico da operação (A/B), apresentaremos o programa ex03.1.asm:

Programa ex03.1.asm

```
    #include <p18f4550.inc>

    #define FA       0x00   ; Endereço de A
    #define FB       0x01   ; Endereço de B
    #define FQ       0x02   ; Endereço de Q(uociente)
    #define FR       0x03   ; Endereço de R(esto)

    #define COUNT    0x04   ; Endereço do contador
    #define SIGNAL_Q 0x05   ; Endereço do sinal final de Q
    #define SIGNAL_R 0x06   ; Endereço do sinal final de R

    #define VA     0xEC     ; Valor de A (-20)
    #define VB     0x06     ; Valor de B (6)

    org 0x0000
    bra start

    org 0x0020
start:
    MOVLW   VA        ; Inicializando variáveis
    MOVWF   FA, 0
    MOVLW   VB
    MOVWF   FB, 0
    CLRF    FQ
    CLRF    FR

    MOVF  FA, W         ; Verifica se é divisão do tipo: 0/X = 0
    BZ out
    MOVF  FB, W         ; Verifica se é divisão do tipo: X/0 = indf
    BZ  out

    MOVLW   0x08        ; Inicializando variáveis auxiliares
    MOVWF   COUNT
    CLRF    SIGNAL_Q
    CLRF    SIGNAL_R
            ; Verificar se FA < 0, se sim, FA <- |VA|
    MOVF    FA, W
    BNN     next_step
```

```
    COMF    FA, F
    INCF    FA, F
    BSF     SIGNAL_R, 7     ; Sinal de R será FA(bit7)
    BSF     SIGNAL_Q, 7
next_step:  ; Verificar se FB < 0, se sim, FB <- |VB|
    MOVF    FB, W
    BNN     end_step
    COMF    FB, F
    INCF    FB, F

end_step:
    ANDLW   0x80
    XORWF   SIGNAL_Q, F    ; Sinal de Q será FA[bit7] XOR FB[bit7]
    MOVFF   FA, FQ
div_loop:
    RLCF    FQ, F          ; Q << 1, carry <- MSB(Q)
    RLCF    FR, F          ; R << 1, LSB(R) <- carry

    BTFSS   FR, 7          ; R < 0 -> R = R + VB
    bra     r_positive

    MOVF    FB, W
    ADDWF   FR, F
    bra     set_qbit
r_positive:                 ; R >= 0 -> R = R - VB
    MOVF    FB, W
    SUBWF   FR, F
set_qbit:                   ; Se R >= 0 -> Q[bit0] = 1, else Q[bit0] = 0
    BTFSS FR, 7
    BSF FQ, 0
    BTFSC FR, 7
    BCF FQ, 0
    DECFSZ COUNT, F         ; Condição de parada (COUNT == 0)
    bra div_loop

    BTFSS FR, 7             ; correção final: se R < 0, R = R + VB
    bra  insert_signal_fQ
    MOVF FB, W
    ADDWF FR, F
insert_signal_fQ:           ; aplicar sinal do quociente
    BTFSS SIGNAL_Q, 7
    bra insert_signal_fR
    COMF FQ, F
    INCF FQ, F
insert_signal_fR:           ; aplicar sinal do resto
    BTFSS SIGNAL_R, 7
    bra out
    COMF FR, F
    INCF FR, F
out:
    bra out
    end
```

Esse programa opera números *signed* e *unsigned*, com um número de iterações de divisão fixa, numa quantidade igual a largura da palavra que compõe o número. Para esse exercício consideremos operar números de 8 bits, assim, cada operação levará 8 iterações para qualquer valor de VA e VB.

Então, para esse programa:

$$(Q, R) = \frac{A}{B}, B \neq 0, A \in \mathbb{Z}, B \in \mathbb{Z}^*$$

O algoritmo utilizado para essa aplicação é o ***Non-Restoring Division***. Esse método de divisão é análogo à divisão manual ensinada no ensino fundamental, em que o processo é conduzido passo a passo sobre os algarismos do dividendo. De forma conceitual, o funcionamento segue a seguinte sequência:

1. Avaliamos o algarismo mais significativo do dividendo.

2. Verificamos se esse valor é maior ou igual ao divisor, caso não seja, trazemos o próximo algarismo e formamos um novo número.

3. No quociente, colocamos o maior valor possível que, multiplicado pelo divisor, resulte em um número menor ou igual ao trecho atual do dividendo.

4. Subtraímos o resultado dessa multiplicação do valor analisado do dividendo, obtendo assim o resto parcial.

5. Em seguida, "baixamos" o próximo algarismo do dividendo ao lado desse resto, formando um novo valor que será novamente operado e o processo se repete até que todos os algarismos tenham sido utilizados.

No contexto binário, o algoritmo ***Non-Restoring Division*** adapta essa mesma lógica. Cada algarismo é substituído por um bit, e o processo de "baixar" torna-se uma rotação de **FR** e **FQ**. Em vez de comparações e multiplicações decimais, o algoritmo realiza operações de soma e subtração em complemento de dois, decidindo, a cada iteração se o resto parcial deve ser restaurado (adição) ou não (subtração). Dessa forma, a divisão é executada *bit* a *bit*, garantindo maior eficiência.

Inicialmente definimos espaços de memória para armazenar variáveis de entrada e saída, como já mencionado no programa anterior, porém nesse caso adicionamos 3 novas variáveis auxiliares: **SIGN_Q**, **SIGN_R** e **COUNT** que serão explicadas ao longo deste relatório.

Antes de entrar no loop de operação, convertemos **VA** e **VB** para seus valores em módulo. A ideia é avaliar os sinais de **A** e **B** e definir previamente os sinais de **Q** e **R**, podendo trabalhar mais facilmente com valores *unsigned*.

Para isso **SIGN_Q** e **SIGN_R**, correspondem respectivamente, ao sinal final de **FQ** e **FR**. Para definir esses sinais com base nos sinais de **A** e **B** podemos entender a seguinte tabela verdade:

A[bit 7]	B[bit 7]	Q[bit 7]	R[bit 7]
0	0	0	0
0	1	1	0
1	0	1	1
1	1	0	1

Como a divisão de sinais iguais, resulta num sinal positivo e sinais diferentes geram sinal negativo, MSB de **FQ** corresponde a operação *XOR* entre MSB de **A** e **B**. Por sua vez, para entender o sinal do resto, consideremos uma parte do **Teorema da Divisão de Euclides:**

$$0 \leq r < |c| \quad \Rightarrow 0 \leq r < c, \; se\, A \geq 0$$
$$-c < \; -r \leq 0, \; se\, A < 0$$

Pelo teorema, o sinal de **FR** é o mesmo de **VA.**

Logo, as etapas pré-operação definem os sinais finais de **FQ** e **FR** e transformam os valores negativos em positivos. O endereço **COUNT** armazena um valor literal 0x08 para o contador que será utilizado para varrer 1 byte.

Iniciando a divisão, rotacionamos **FQ** e **FR**, verificamos de **FR** é negativo, se sim somamos **FB** (divisor) em **FR**, se não, subtraímos. Em seguida, se **FR** for maior ou igual a 0, o LSB de **FQ** é setado, caso contrário, é resetado.

Por fim, definimos o sinal de **FQ** e **FR** com base **SIGN_Q** e **SIGN_R** definidos no início.

A Figura 6 mostra, ao fim do programa, os endereços de memória com os valores de resultado da divisão de *0xEC* (-20) pro *0x06* (6), espera-se que *(FQ, FR) = (0xFD, 0xFE) = (-3, -2)*:

Address	00	01	02	03	04	05	06
000	14	06	FD	FE	00	80	80
010	00	00	00	00	00	00	00
020	00	00	00	00	00	00	00
030	00	00	00	00	00	00	00
040	00	00	00	00	00	00	00

Figura 6:. Em azul, os endereços FQ e FR com os valores esperados